VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

BASIC PROGRAMMING

Lecture 7: Data Structures and File I/O

Dr. NGUYEN Hoang Ha
Dr. NGUYEN Minh Huong

Agenda

- Data Structures
  - Fundamental
  - Structure and Functions
  - Union
- File Input and Output

Fundamental about Structures

Why Structure?

- How to keep different data aspects of items together?
- Examples:
  - Track students of Basic Programming class with their attributes:
    - Student id
    - Name
    - Mark
  - Manage properties of each book:
    - Title
    - Author
    - Subject
    - Book ID
    - Publishing year

Structures

- A user-defined data type
- Combine a limited number of data elements into a unified data unit (corresponding to a record in general)
- Data elements are in arbitrary types
- Example:

  A record about a book in a library has properties:

  - Title
  - Author
  - Subject
  - Book ID
  - Publishing year

  → a structure named Book

Declaring structure

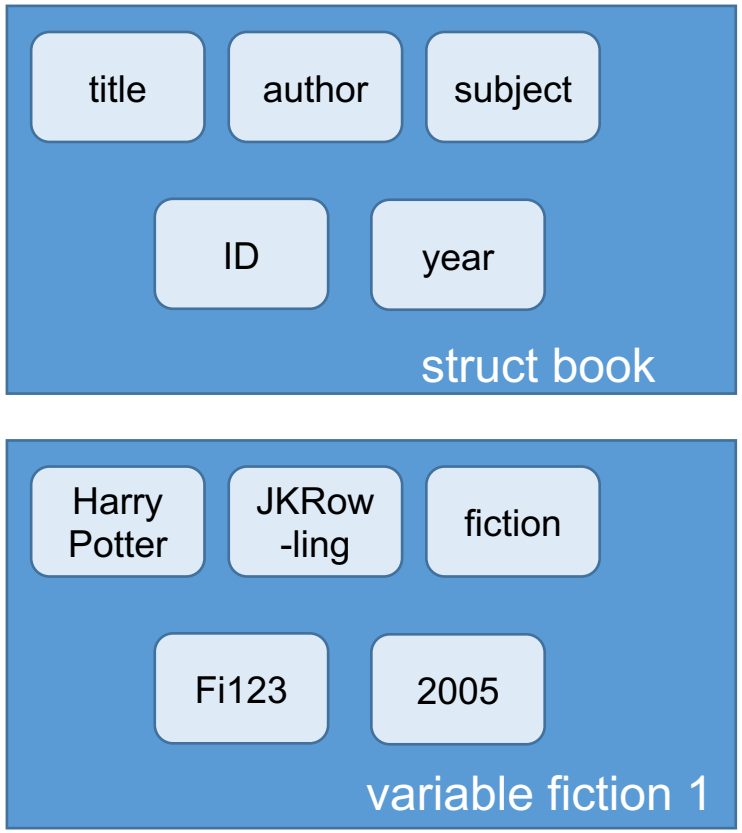
- Syntax:
  - Declaring a struct

```
struct Book
{
    char* title;
    char* author;
    char* subject;
    char* ID;
    int year;

};
```

  - Declaring a struct **variable**

```
struct Book fiction1;
```



Initializing Structures

```
struct Book fiction1=
{

"Harry Potter", "JKRowling" , "fiction" , "Fi123", 2005

};
```

Accessing Structures

- Accessing a member of a struct: "." operator

  variable_name.attribute

Access to fiction1's title

```
struct Book fiction1;
fiction1.title = "Harry Potter";
fiction1.author = "JKRowling";
fiction1.subject = "fiction";
fiction1.ID = "Fi123";
fiction1.year = 2005;
```

Accessing Structures

```
…
printf("The book fiction1 information: \n");
printf("Title: %s \n", fiction1.title);
printf("Author: %s \n", fiction1.author);
printf("Book ID: %s \n", fiction1.ID);
printf("Publishing year: %d \n", fiction1.year);
```

Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Book {
    char* title;
    char* author;
    char* subject;
    char* ID;
    int year;
};
int main( ) {
    struct Book fiction1 = {"Harry Potter", "JKRowling" , "fiction" ,
    "Fi123", 2005};
    struct Book book1;/* Declare book1 of type Book */
    book1.title = "C Programming";
    book1.year = 6495407;
    ....
}
```

typedef

- How to declare 5 books has the same struct?

```
struct Book fiction1;
struct book fiction2;
struct book fiction3;
struct book fiction4;
struct book fiction5;
```

- typedef: give a shorter name for the struct

```
typedef struct book bk;
bk fiction1;
bk fiction2;
…
```

typedef

- Another way to use typedef:

```
typedef struct
{
    char* title;
    char* author;
    char* subject;
    char* ID;
    int year;

} Book;

Book fiction2;
Book fiction3;
Book fiction4;
```

Size of a struct

```
Book fiction1=
{
"Harry Potter", "JKRowling" , "fiction" , "Fi123", 2005
};

printf("Size of book struct is %lu \n", sizeof(Book));
```

Notes

- Struct variables cannot be compared
- Members of struct variables can be compared

Arrays of structs

- Declaration:

```
Book fiction[5];
```

- Accessing to members of struct elements

```
Book fiction[5];
fiction[1].title = "The Little Prince";
fiction[1].author = "Antoine de Saint-Exupéry";
```

Pointers to Structures

- A pointer to a structure:
  - contains address of the structure

```
Book* p = &fiction1;
```

  - Can be used to access to member of struct:

```
printf("Title of the book is %s \n", (*p).title);
```

Or

```
printf("Title of the book is %s \n", p->title);
```

Operations with structures

- Structure member can be used as any other variables
- Assignment:

```
Book fiction2 = fiction1;
```

- "&" operator:

```
&fiction1 // return address of the struct variable fiction1
```

- Cannot add, compare or perform any other operations with struct variables

→ Write your own functions

Pointer to Struct

- Consider Struct as other built-in datatypes when declare a pointer to a struct variabble

```
struct Book fiction1 = {"Harry Potter", "JKRowling" ,
"fiction" , "Fi123", 2005};

struct Book* book_ptr = &fiction1;
```

- And access member of Struct via the pointer:

```
printf( "Book 1 title : %s\n", (*book_ptr).title);
```

- Or a more intuitive way

```
printf( "Book 1 title : %s\n", book_ptr->title);
```

Dynamic Memory Allocation for Structures

```
struct Book* tech1_ptr = (struct Book*)malloc(sizeof(struct Book));
tech1_ptr->title = "Programming in C (4th Edition) ";
tech1_ptr->author = "Stephen Kochan";
tech1_ptr->subject = "Basic programming";
tech1_ptr->ID = "ProgC4";
tech1_ptr->year = 2014;


...


free(tech1_ptr);
```

Passing a Struct to a Function

- Like other datatype, Struct can play as a parameter of a function

```
void print_book(struct Book aBook){
    printf( "Book title : %s\n", aBook.title);
    printf( "Book author : %s\n", aBook.author);
    printf( "Book subject : %s\n", aBook.subject);
    printf( "Book book_id : %s\n", aBook.ID);
}
```

- Recall: the parameter is passed by value → the function cannot change the original variable

Returning a struct from a function

```
struct Book enter_book(){
    struct Book aBook;
    printf("Enter the book title and year ");
    aBook.title = malloc(100);
    scanf("%s %d", aBook.title, &aBook.year);
    return aBook;
}
```

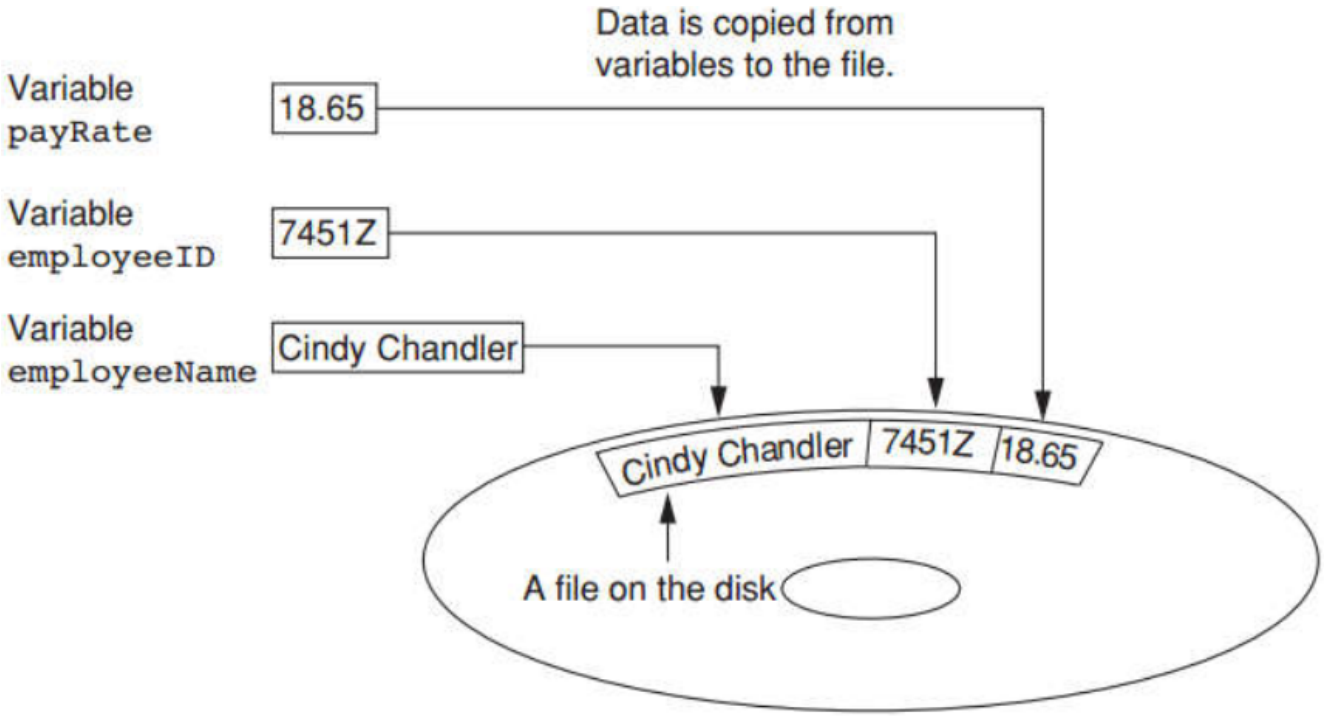
File Input and Output

Data and Permanent Storage

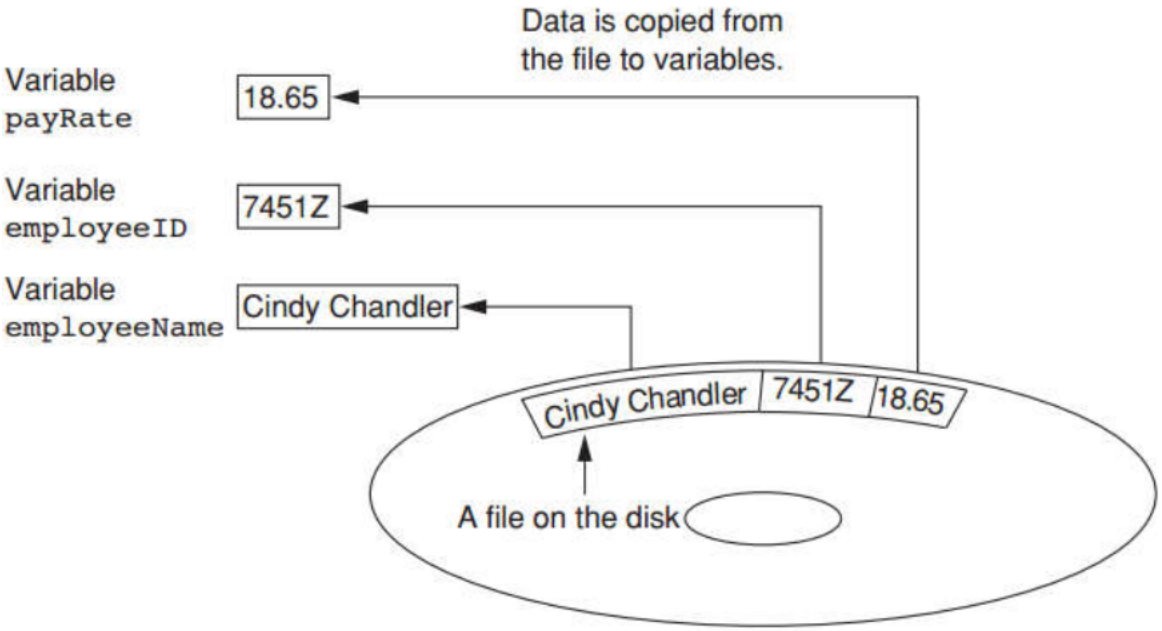
- Data kept in variables is stored in RAM and disappears once the program stops running
- →A program needs to save data for later use → it writes the data to a file in hard disk

- The data can then be read from the file at a later time

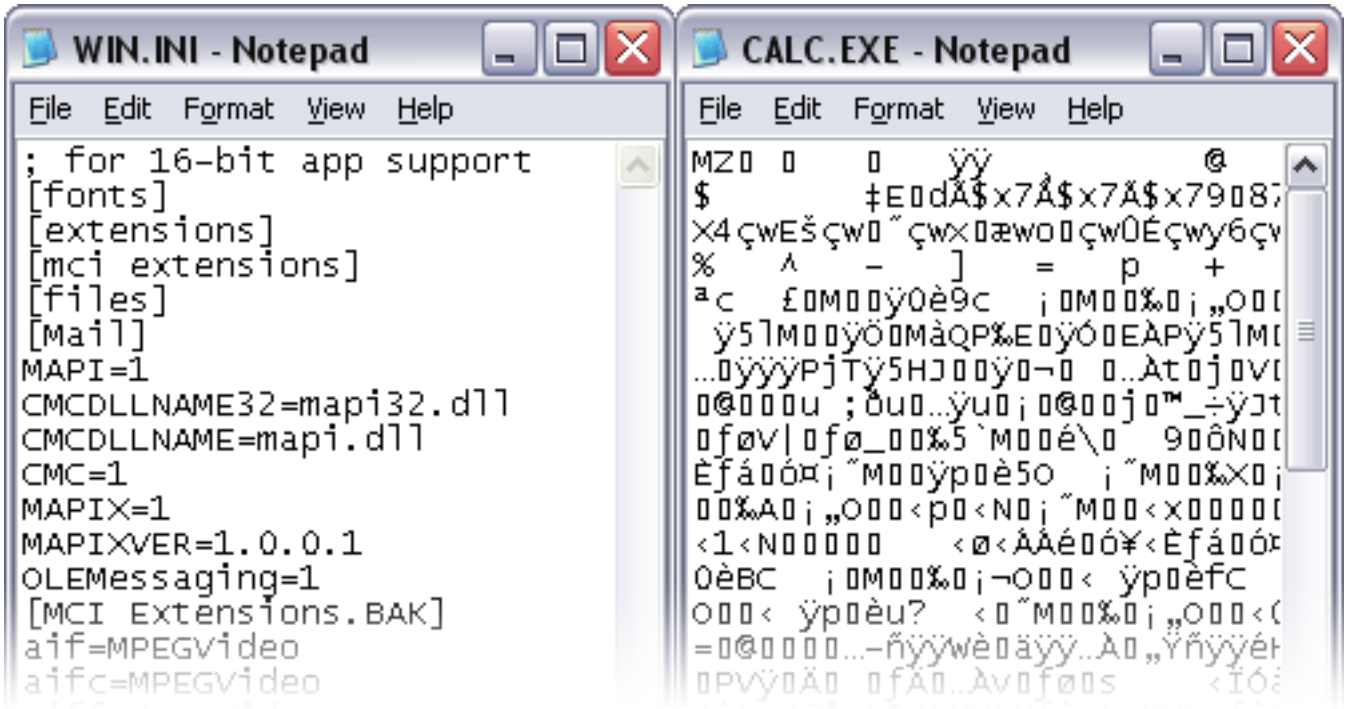
Writing Data to a File



Reading Data from a File

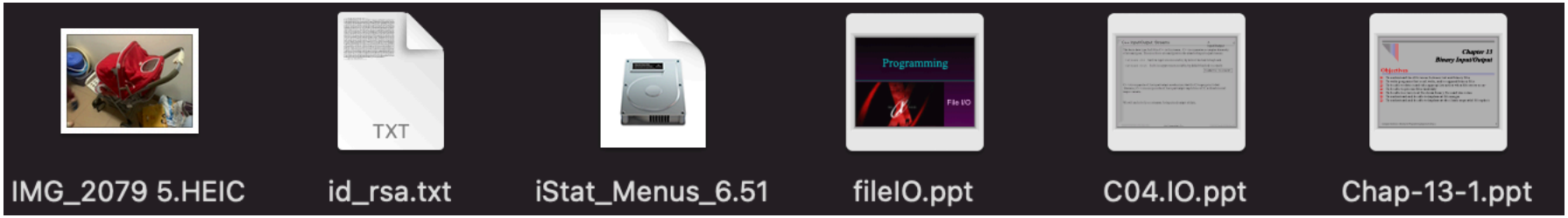


Text File vs. Binary File



Filename

- Files on disk are identified by their filename
- Filenames must be unique in a folder
- Filename extensions are short sequences of characters at the end of a filename and after the last dot character
- Filename extensions usually indicate the type of data stored in the file



Text File vs. Binary File

Text

- Data encoded as text using scheme such as ASCII or Unicode
- Can be viewed by text reader applications (Notepad)

Binary

- Data in pure format (not yet encoded)
- Cannot be viewed by text reader application

Procedure to Work with a File

- Step 1: Open the file
  - To create a connection between the program and the file
  - Opening an output file → create a file on a disk, allow program to write on it
  - Opening an input file → allow program to read data from the file
- Step 2: Process the file
  - To write (or read) the output (or input) file
- Step 3: Close the file
  - To disconnect the file from the program

File IO Stream

- In C all input and output is done with streams
- Stream is nothing but a sequence of bytes of data
- A sequence of bytes flowing into the program is called input stream
- A sequence of bytes flowing out of the program is called output stream

Opening-File modes

Mode	Description
r	Opens an existing text file for reading purpose.
w	Opens a text file for writing. If it does not exist, then a new file is created. Here your program will start writing content from the beginning of the file.
a	Opens a text file for writing in appending mode. If it does not exist, then a new file is created. Here your program will start appending content in the existing file content.
r+	Opens a text file for both reading and writing.
w+	Opens a text file for both reading and writing. It first truncates the file to zero length if it exists, otherwise creates a file if it does not exist.
a+	Opens a text file for both reading and writing. It creates the file if it does not exist. The reading will start from the beginning but writing can only be appended.

Reading a Text File

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    char ch;
    FILE *fp = fopen("content_to_be_read.txt", "r"); // read mode
    if (fp == NULL)
    {
        printf("Error while opening the file.\n");
        exit(EXIT_FAILURE);
    }

    printf("The contents:\n");

    while((ch = fgetc(fp)) != EOF)
    printf("%c", ch);

    fclose(fp);
    return 0;
}
```

Writing a Text File

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *out_file = fopen("out.txt", "w"); // write only
    fprintf(out_file, "this is the content written out %d\n", 2021);
    return 0;
}
```